

A 1.4.1 Interpreters and Compilers (HL Only)



1.4.1 Evaluate the translation processes of interpreters and compilers.

A computer cannot run Python or Java source code directly. The processor understands only machine code, so every program written in a high-level language has to be translated first, and there is more than one way to carry out that translation. The approach chosen decides how fast the program runs, how long you wait before you can test a change, whether the same program will run on a different machine, and when you find out that something is broken. This page sets out how interpreters and compilers work, adds the two hybrid approaches the syllabus names (just-in-time compilation and bytecode interpreters), compares all four on error detection, translation time, portability and applicability, and then weighs which approach suits rapid development, performance-critical work and cross-platform development.

Last Updated: 14 June 2026

Key Questions

- How does an interpreter differ from a compiler in the way it translates and runs a program?
- When does each approach detect errors, and why does that matter while you are writing code?
- Where do just-in-time compilation and bytecode interpreters sit between the two, and what do they buy you?
- Given a situation (a quick prototype, a high-performance game, a cross-platform app), which translation approach would you choose, and why?

IB Command Term note

The command term is **evaluate**. An evaluate question asks you to reach a judgement by weighing strengths against limitations, not simply to list features.

A strong answer to "Evaluate the use of an interpreter for situation X" says what the interpreter does well there, says where it falls short, and then commits to a reasoned conclusion that fits the

situation given. Accurate description on its own will not reach the top marks. Tie every point to the scenario: the right translation approach for a research prototype is not the right one for a shipped game.

THINK FIRST

You save a short Python script and it runs the instant you press enter. A C program of the same length needs a separate build step before it will run at all. Before reading on, work out what that build step is doing, and where the work it does goes in the Python case, since the work itself does not disappear.

How does an interpreter work?

An **interpreter** reads your source code and carries out its instructions directly, working through the program one statement at a time. It translates each statement into machine operations and runs it immediately, then moves to the next. There is no separate finished file at the end: the interpreter does the translation afresh every time the program runs, which is why the interpreter itself has to be installed on any machine that runs the code.

Because translation and execution are combined, you get output the moment you run the program, with no build to wait for. A statement inside a loop is re-translated on every pass, so execution is slower than it needs to be. Python, Ruby and JavaScript are the languages students meet most often here.

You change one line in a 30-line Python data-cleaning script and run it again. It executes straight away, you read the result, you change another line. That tight edit-run-look loop is exactly what an interpreter is good at.

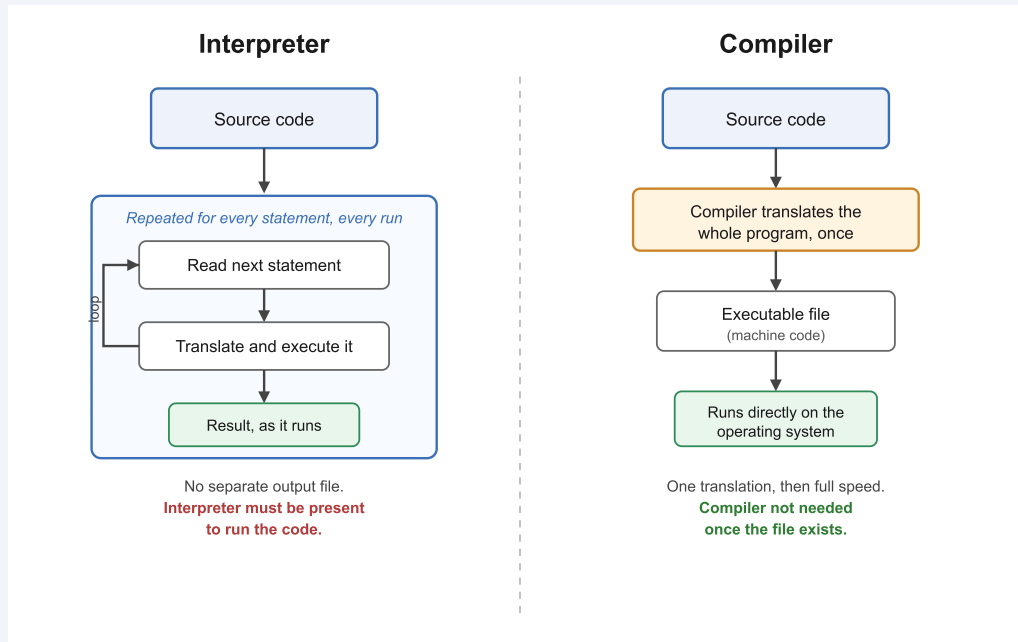
How does a compiler work?

A **compiler** translates the whole program in one pass, before it runs at all. It takes your source code as input and produces a separate file of machine code, the executable, which the operating system can then run on its own. Once that file exists the compiler is finished with: you can delete it, or send the executable to someone who has never installed it, and the program still runs.

That translation is where compilers are better for speed. The compiler can examine the whole program at once, rearrange and optimise the machine code, and settle once anything an interpreter would have to rework on every run. The program then executes as native machine code at the full

speed of the hardware. Every change means recompiling before you can test, and on a large project that wait is real. C, C++ and Go are typical compiled languages.

A studio building a 3D game compiles it once into a native executable for each platform it ships on. Players run that executable directly. None of them needs a compiler installed, and the rendering loop runs as fast as the hardware allows.



An interpreter translates and runs each statement repeatedly as the program executes; a compiler translates the whole program once into a standalone executable.

Where do JIT compilation and bytecode interpreters fit?

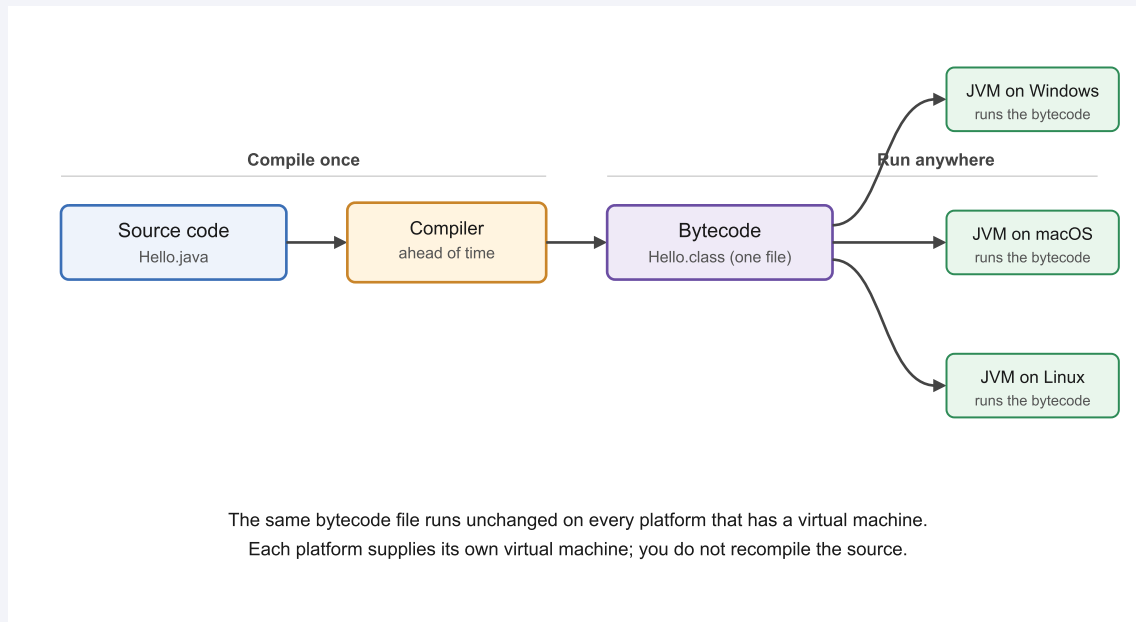
The syllabus names two methods that combine their qualities, and you are expected to compare all four.

Bytecode interpreters

A bytecode interpreter splits the job into two stages. First the source is compiled, ahead of time, into bytecode: a compact, low-level instruction set that is not tied to any one processor. Then a virtual machine, which is itself an interpreter, executes that bytecode. Java is the standard example. The `javac` compiler turns `Hello.java` into `Hello.class` bytecode, and the Java Virtual Machine (JVM) runs that bytecode on whatever computer it is installed on.

The result is portability without losing all the benefit of compilation. The same `.class` file runs unchanged on Windows, macOS or Linux, because each platform supplies its own JVM. Bytecode is also closer to machine code than source is, so the virtual machine executes it faster than it could

interpret the original source line by line. This is why calling Java simply "compiled" or simply "interpreted" is wrong: it is compiled to bytecode, and that bytecode is then executed by a virtual machine. CPython works the same way, compiling Python to bytecode that its own virtual machine runs.



Source is compiled once into platform-independent bytecode; each operating system runs that same bytecode through its own virtual machine.

Just-in-time (JIT) compilation

JIT compilation goes one step further. A virtual machine starts by interpreting bytecode, which lets the program begin quickly, and watches which parts run most often. The sections that run repeatedly, the "hot" code, are compiled to native machine code during execution and reused from then on.

The program keeps the quick start and portability of bytecode interpretation but reaches close to compiled speed on the code that runs most. Modern JVMs and the V8 engine that runs JavaScript in browsers both work this way.

How do the approaches compare?

These four dimensions matter for an evaluate question, and they are the four the syllabus lists.

Error detection. A compiler checks the whole program before producing an executable, so it catches whole classes of error (syntax mistakes, type mismatches, undeclared names) before the program ever runs. An interpreter only meets an error when execution reaches the offending statement, so a program can run for several seconds and then fail on a line that a particular input finally exercised.

Neither is strictly better: the compiler tells you sooner, the interpreter tells you exactly which run-time situation broke. Bytecode verification sits between them, catching some malformed bytecode before the virtual machine runs it.

Translation time and execution speed. Keep these two apart, because confusing them is the single most common error on this topic. Translation time is how long you wait before the program can start; execution speed is how fast it then runs.

An interpreter has almost no translation time, so it starts at once, but its execution is slower because translation continues throughout the run. A compiler is the reverse: a noticeable wait while it translates everything, then fast native execution every time afterwards.

Portability. Interpreted and bytecode programs are portable: the same source, or the same bytecode, runs anywhere the interpreter or virtual machine is installed. Native compiled code is not. An executable built for 64-bit Windows will not run on macOS or on an ARM processor, so you must recompile for each target.

JIT sits in the middle, because it can tune the machine code it generates to the exact platform it finds itself on.

Applicability. This follows from the three above and is best seen through concrete scenarios, which the next section sets out.

COMMON EXAM MISTAKE

Writing that "interpreters are faster than compilers" without saying faster *at what*. Interpreters are faster to **start**, because there is no build step. Compiled programs are faster to **run**. An answer that does not separate translation time from execution speed reads as a misunderstanding and will not earn the comparison marks.

WORKED EXAMPLE

Take a program in which lines 1 to 7 are valid, and line 8 tries to add a number to a piece of text. Watch where the error surfaces.

Statically typed compiled language (for example Java): the compiler examines the whole program first. It rejects the type error on line 8 at compile time and produces no executable, so lines 1 to 7 never run. You fix the mistake before anything executes at all.

Dynamically typed interpreted language (for example Python): the interpreter runs lines 1 to 7, producing their output, then halts at line 8 with a run-time error when it reaches that statement. You have seen partial output, which can help you see the fault in context, but the program ran in a broken state up to that point.

The same fault, found at two different moments. That difference in *when* is the heart of the error-detection comparison.

Below is a reference comparison across the four translation processes. Use it for revision rather than reading it straight through.

	Interpreter	Compiler	JIT compiler	Bytecode interpreter
Error detection	At run time, as each statement is reached	At compile time, before execution	At run time; verification can catch some errors first	At run time; bytecode verification catches some errors first
Translation time	Negligible; starts at once	Slow; whole program translated upfront	Fast start, then compiles hot code during the run	Faster than full compilation; bytecode is an intermediate step
Execution speed	Slower; translation continues throughout	Fast; optimised native code	Near-compiled on hot paths	Faster than direct interpretation, below native
Portability	High; runs anywhere the interpreter exists	Low; native code is platform-specific	Moderate; tuned to the host at run time	High; same bytecode runs on any compatible VM
Typical use	Scripting, prototyping, dynamic languages	Performance-critical and systems software	Balance of portability and speed	Portable apps with moderate performance needs
Example languages	Python, Ruby, JavaScript	C, C++, Go	Java (HotSpot), JavaScript (V8)	Java, Python (CPython)

Which approach suits which job?

Rapid development and testing

When you are writing and testing something new, the wait between a change and a result is what slows things down. An **interpreter** removes that wait: no build step, so you edit, run, and see the

outcome in one tight loop, and you can test fragments interactively. For prototyping an algorithm or scripting a quick task, that immediate feedback outweighs slower execution, because the program is small and short-lived. Here the **interpreter** is the better choice.

Performance-critical applications

When the program must run as fast as the hardware permits (a 3D game, a physics simulation, an operating system component) the upfront cost of compilation is worth paying many times over. A **compiler** can optimise the whole program and emit native code that runs at full speed on every later execution. For managed environments that still need speed, a **JIT compiler** reaches much of the same performance while keeping portability. The slow build matters far less than the run-time speed, so a compiled approach wins.

Cross-platform development

When the same program must run on several operating systems, recompiling and distributing a separate native executable for each one is awkward. A **bytecode interpreter** avoids it: compile once to bytecode and ship that single file, and any machine with a compatible virtual machine runs it. **JIT** adds speed on top, tuning the code to each host at run time. For "write once, run anywhere", the bytecode approach is the natural fit.

Summary

Use this summary to revise the main points for the section.

Summary			
Approach	What it does	Error detection	Strongest for
Interpreter	Translates and runs each statement as the program executes	Run time	Rapid development and testing
Compiler	Translates the whole program once into a native executable	Compile time	Performance-critical applications
Bytecode interpreter	Compiles to portable bytecode that a	Run time, plus bytecode verification	Cross-platform development

Approach	What it does	Error detection	Strongest for
	virtual machine runs		
JIT compiler	Interprets bytecode, then compiles hot code to native at run time	Run time	Balance of speed and portability

Two key ideas to remember: **interpreters** are quick to *start* and slow to *run*; **compilers** are the reverse. Portability and native speed pull against each other, and **bytecode** plus **JIT** is the engineering compromise between them.

EXAM TIP

In a comparison, name the dimension before you compare on it ("On translation time...", "On portability..."). It signals to the examiner the exact basis of each point and stops your answer drifting into a vague list. For an evaluate question, finish with a judgement that names the scenario.

Try it on paper

The site quiz below covers the multiple-choice recall. These two written questions need a fuller response, so work them on paper, then check against the model answers.

1. Explain why compiled code generally executes faster than interpreted code. **[2 marks]**

Model answer. A compiler translates the whole program into optimised machine code before it runs, so at run time the processor executes native instructions directly [1]. An interpreter translates each statement as it runs and repeats that work on every pass through a loop, which adds overhead during execution and so runs more slowly [1].

2. A student is developing and testing a new sorting algorithm and wants to try changes quickly. Evaluate the use of an interpreted language rather than a compiled one for this task. **[4 marks]**

Model answer.

For: An interpreter has no build step, so the student sees the result of each change immediately and can test the algorithm interactively, which suits fast, repeated iteration [1]; errors surface at the exact statement reached during a run, helping locate faults in the logic while developing [1].

Against: Interpreted execution is slower, so timing the algorithm's true performance is unreliable [1];

if the algorithm is later deployed where speed matters, a compiled language would be needed, so the interpreted version is a development aid rather than the final form [1].

(Full marks need a clear judgement, for example: for development and testing the interpreter is the better choice because iteration speed matters more than execution speed at this stage.)

Quiz

1

Which of the following best describes the role of an interpreter?

- A. ☐ Optimises code for specific hardware architectures.
- B. ☐ Generates an executable file that can be run independently.
- C. ☐ Translates the entire program into machine code at once.
- D. ☒ Translates and executes code line by line.

An interpreter works through the program statement by statement, translating and running each before moving on. The interpreter produces no standalone file and must be present whenever the code runs.

2

What is a key advantage of using a compiler?

- A. ☐ Faster translation time.
- B. ☐ Enhanced portability across different platforms.
- C. ☐ Easier debugging of runtime errors.
- D. ☒ Improved execution speed of the program.

Compilation produces optimised native code that runs quickly every time.

3

Which translation method is most suitable for rapid prototyping and scripting?

- A. ☒ Interpreter
- B. ☐ JIT compiler
- C. ☐ Compiler
- D. ☐ Assembler

An interpreter gives immediate feedback with no build step, which suits quick, repeated testing. A compiler imposes a build wait on every change. A JIT compiler helps run-time performance rather than the edit-test loop, and an assembler translates assembly language, not a high-level scripting language.

4

What is the primary benefit of using bytecode interpreters?

- A. ☐ Reduced memory usage.
- B. ☒ Platform independence.
- C. ☐ Simplified debugging process.
- D. ☐ Maximum execution speed.

The same bytecode runs on any platform with a compatible virtual machine, so the program is portable.

5

JIT compilation is a hybrid approach that combines the benefits of:

- A. ☒ Interpreters and compilers.

- B. ☐ Assemblers and loaders.
- C. ☐ Compilers and linkers.
- D. ☐ Interpreters and assemblers.

JIT keeps the quick start and portability of interpretation while compiling frequently run code to native speed, so it draws on both interpreters and compilers. The other pairs name unrelated stages of the toolchain.

6

Which of the following is NOT a characteristic of compiled code?

- A. ☐ Often platform-specific.
- B. ☐ Can undergo extensive code optimisation.
- C. ☐ Requires the compiler to be present during execution.
- D. ☐ Generally executes faster than interpreted code.

Once compiled, the executable runs on its own; the compiler is not needed at run time (that requirement belongs to interpreters). The others are all genuine characteristics of compiled code.

7

Error detection in interpreters typically occurs:

- A. ☐ During program execution, as each statement is reached.
- B. ☐ After the program has finished running.
- C. ☐ Before any code runs, during a separate checking phase.
- D. ☐ Only when the source file is saved.

An interpreter reports an error at the moment it reaches the faulty statement during the run.

8

Which translation method is commonly used for languages like Java and Python (CPython)?

- A. ☐ Direct interpretation
- B. ☐ Just-in-time compilation
- C. ☒ Bytecode interpretation
- D. ☐ Ahead-of-time compilation

Both compile source to bytecode that a virtual machine then runs. JIT can be layered on top of bytecode interpretation, as in the JVM, but the underlying method is bytecode interpretation.

9

In which scenario would a compiler be the most suitable choice?

- A. ☐ Building a web application with dynamic content.
- B. ☐ Prototyping a new algorithm for research purposes.
- C. ☐ Developing a small script for automating tasks.
- D. ☒ Creating a high-performance game with demanding graphics.

A game needs fast, optimised native execution, which a compiler provides. The others all favour quick iteration or portability, where an interpreter or bytecode approach fits better.

10

What does portability refer to in the context of code translation?

- A. ☐ The speed at which code can be translated.

B. ☐ The efficiency of code in terms of memory usage.

C. ☒ The ability of code to run on different operating systems and hardware.

D. ☐ The ease with which code can be moved between different storage devices.

Portability is whether the same program runs across platforms without being rewritten or recompiled.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**